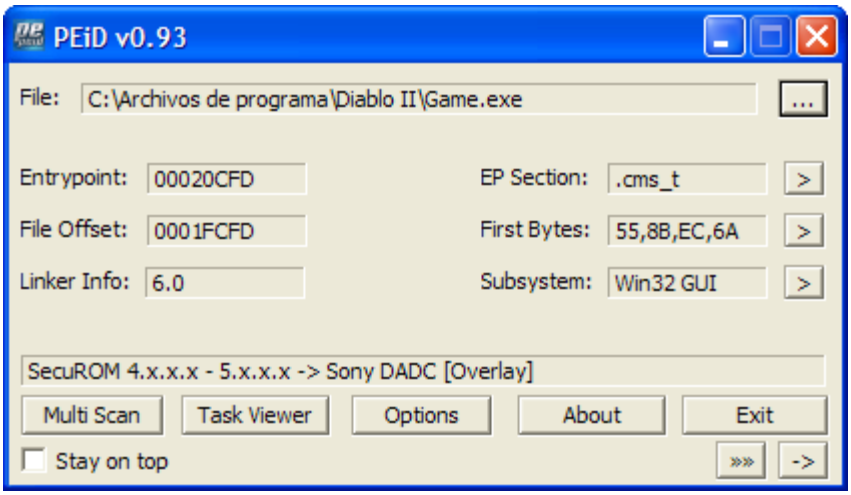
Estudio de SecuROM v4.x.x.x - 5.x.x.x

Programa:	Diablo II_Lord of Destruction
Protección:	SecuROM v4.x.x.x - 5.x.x.x
Objetivos:	Desempakar y conseguir que corra
URL descarga:	http://blizzard.com
Dificultad:	Un poco complicado
Herramientas:	OllyDbg v1.10 + PEiD v0.92 + ImpREC v1.6 + DAEMON Tools v3.44 + ConTEXT v0.98 + Hex WorkShop 4.23 + PEditor v1.7 + LordPE Deluxe
Cracker:	SyXe'05 .::[hAcK-c0d3d]..[CracksLatinoS]::. Fecha: 05-02-06

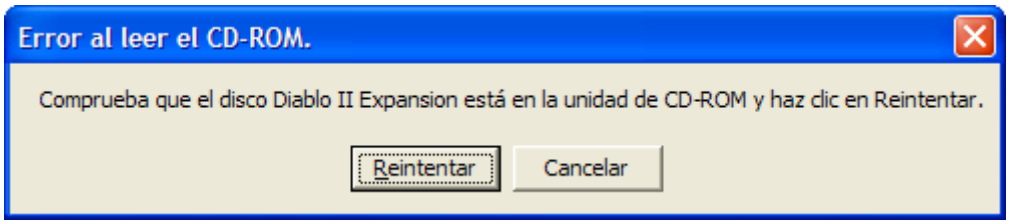
Un saludo a todos CracksLatinoS!

Después de mucho tiempo sin escribir nada volvemos otra vez para mirar de estudiar y desempakar una de las protecciones comerciales más comunes en CD de juegos, se trata de SecuROM en su versión 4 ó 5 pues PEiD no lo especifica, el estudio se centrará sobre el juego DiabloII al que se le ha agregado la expansión "Lord of Destruction" aunque esto da lo mismo ya que el ejecutable no varía de una versión a otra, una vez instalado el programa echamos un vistazo al ejecutable Game.exe con PEiD v0.92 y vemos esto:



Hasta ahora lo dicho, pero… cómo atacamos esta protección ¿? Antes de comenzar nos hacemos con una copia de las DAEMON Tools, yo estoy utilizando la versión 3.44 para este tute y me funciona sin problemas, si alguno no la tiene o no sabe donde bajarla (cosa que dudo) que me lo diga y la subo a algún FTP o similar.

Vamos al tajo… si intentamos correr el juego nos encontramos con esto:

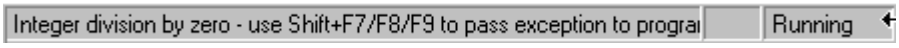


Parece el típico CD-Check (comprobación de CD en su unidad de CD/DVD) esto significa que si no tenemos el CD original insertado el juego NO correrá, esto es a causa de la protección a abatir "SecuROM", he de decir que a día de hoy todavía no me he enfrentado a esta, espero que me sepan disculpar los posibles errores u omisiones en la forma y contenido, así cómo por la información aquí dada, para cualquier comentario y/o corrección podéis contactar conmigo:

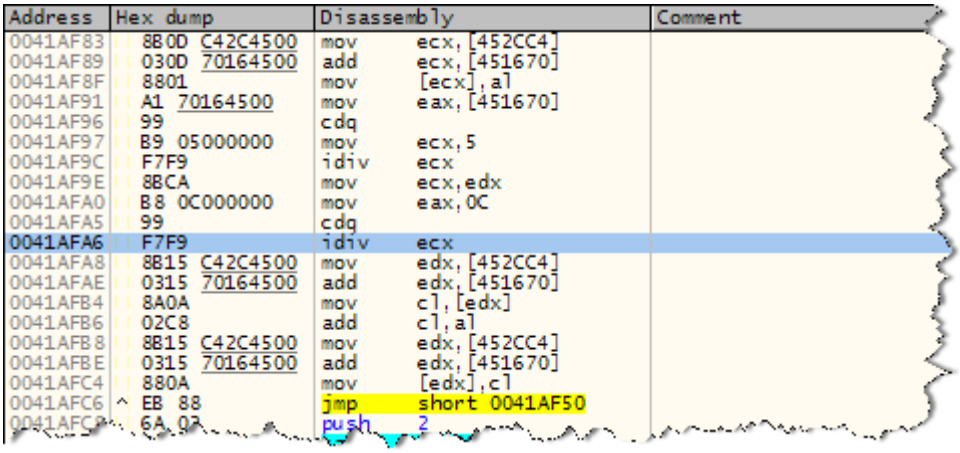
syxe05@gmail.com

Aquí estaré disponible para futuras consultas sobre el tema (o también para cualquier otra cosa concerniente al "cracking") xDD

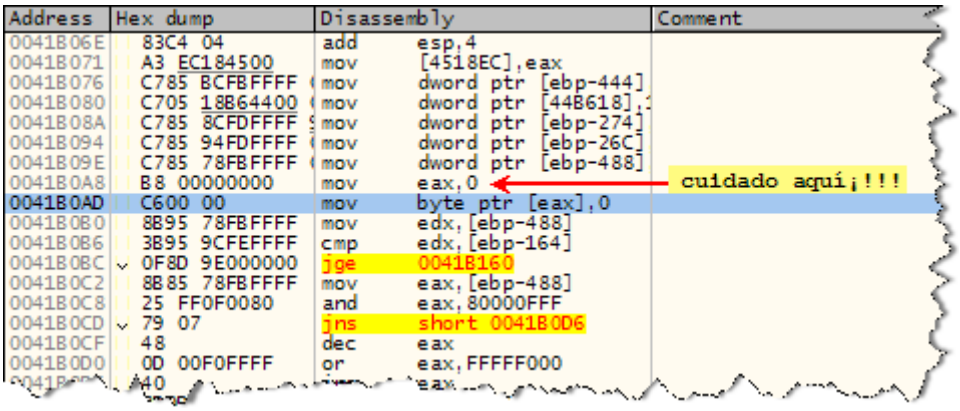
Si cargamos el .exe en el Olly y lo lanzamos queadrá en una excepción parado:



Si pausamos Olly con F12 caeremos en el código donde se produce la excepción:



Ahí se produce la excepción tipo "INTEGER_DIVISION_BY_ZERO" la división por cero de toda la vida vaya¡!! Y estamos además en un loop lo que hace que salte la excepción infinidad de veces, al final sale del loop y continúa desencriptando código, pero luego llegamos a otra zona conflictiva:



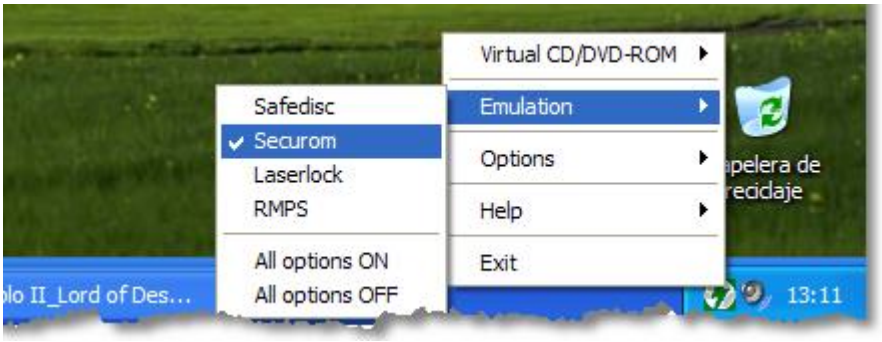
Genera otra excepción voluntariamente, en esta ocasión un "ACCESS_VIOLATION" si miramos la pila veremos donde tiene instalado el manejador de excepciones:

Address	Value	Comment
0012F7B0	0012FFB0	Pointer to next SEH record
0012F7B4	004150C0	SE handler
0012F7B8	77D121AE	USER32.77D121AE
0012F7BC	0012CD34	
0012F7C0	7FFDF000	
0012F7C4	004150B0	Game.004150B0
0012F7C8	00210020	

Vayamos a esa dirección y pongamos un BP con F2, luego apretamos F9 para caer en él, si traceamos desde aquí veremos que nos retorna otra vez al mismo punto y ahora volvemos a caer en el SEH pero a causa de un "SINGLE_STEP" vuelta a tracear y al final nos deja una instrucción después, es decir, en :0041B0B0 si tenemos los registros de depuración a la vista veremos que además nos borra DR1 y DR2, en DR3 no tenía ninguno y por eso no me di cuenta si los borraba o no, veamos cómo quedaron:

```
Debug registers
O O  LastErr ERROR_SUCCESS (00000000)
EFL 00010312 (NO,NB,NE,A,NS,PO,GE,G)
DR0 0041B0B3
DR1 00000000
DR2 00000000   <-- HBPs borrados :(
DR3 00000000
DR6 FFFF0FF0
DR7 00000401
```

Investigando un poco se llega a la conclusión de que también borra DR3, esto nos quiere decir que debemos tener cuidado con el SEH que instala. Poco después entra en un loop sin salida y Olly queda "running" sin que pase nada.
Antes de continuar he de decir que no cuento con una copia original de Diablo II y es por ello por lo que se niega a correr, para ello debería estar el CD original insertado.
Ante en esta situación una opción que tenemos es la de instalar las DAEMON Tools v3.44 y seleccionar la emulación de SecuROM como se ve en la imagen siguiente:



Ya está preparado para emular la protección, si ahora corremos el juego vemos que ahora ya funciona, esto es así porque así conseguimos emular la protección, he de decir también que el ejecutable con el que estoy trabajando es un crack que funciona sin el CD original pero sigue estando protegido, vamos a ver si podemos desempakar esto xDD

Lo siguiente que haremos será cargar el ejecutable protegido en el Olly y debuggearlo a ver si corre, lo cargamos y quedamos detenidos en su Entry Point:

```
Address   Hex dump        Disassembly                          Comment
00420CFD  55              push   ebp
00420CFE  8BEC            mov    ebp,esp
00420D00  6A FF           push   -1
00420D02  68 F8EC4400     push   0044ECF8
00420D07  68 1C394200     push   0042391C
00420D0C  64:A1 00000000  mov    eax,fs:[0]
00420D12  50              push   eax
00420D13  64:8925 000000  mov    fs:[0],esp
00420D1A  83EC 10         sub    esp,10
00420D1D  53              push   ebx
00420D1E  56              push   esi
00420D1F  57              push   edi
00420D20  8965 E8         mov    [ebp-18],esp
00420D23  FF15 A8534500   call   [<&kernel32.GetVersion>]     kernel32.GetVersion
00420D29  33D2            xor    edx,edx
00420D2B  8AD4            mov    dl,ah
00420D2D  8915 58F34400   mov    [44F358],edx
00420D33  8BC8            mov    ecx,eax
00420D35  81E1 FF000000   and    ecx,0FF
```

Ese es el punto de entrada del programa protegido, si presionamos F9 vemos que el juego arranca así que vamos bien, como bien digo si tuviera el CD original me evitaría de utilizar las DAEMON Tools, hay que tener esto en cuenta para el que si que lo tenga pues no le hará falta, lo siguiente será descubrir el punto de entrada real, es decir, el que tenía antes de pasarle SecuROM.

Buscando el OEP xDD

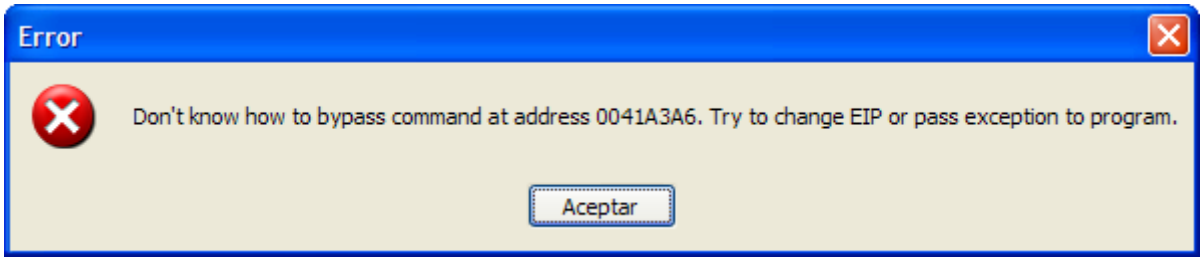
Si desactivamos las excepciones desde las "Debugging Exceptions" y dejamos correr libremente el programa con F9 el programa corre bien, si vamos a la ventana Log pulsando Alt+L veremos todas las excepciones que se sucedieron antes de arrancar:

```
Address   Message
0041AFA6  Integer division by zero
0041AFA6  Integer division by zero
0041AFA6  Integer division by zero
0041AFA6  Integer division by zero
0041B0AD  Access violation when writing to [00000000]
0041B3D4  Access violation when writing to [00000002]
78090000  Module C:\WINDOWS\WinSxS\x86_Microsoft.Windows.Common-Controls_6595b641
200091D6  Access violation when writing to [20000000]
0041CB02  Access violation when writing to [00000005]
004193F3  Access violation when writing to [00000001]
004193F3  Access violation when writing to [00000001]
004193F3  Access violation when writing to [00000001]
004193F3  Access violation when writing to [00000001]
004186B8  Access violation when writing to [00000003]
77E5D342  New thread with ID 00000410 created
719D0000  Module C:\WINDOWS\System32\mswsock.dll
77E53887  Exception 000006BA
76EE0000  Module C:\WINDOWS\System32\DNSAPI.dll
77E53887  Exception 000006BA
```

La última ocurrió en :004186B8 podemos probar a poner un HBP allí y correr de nuevo el programa a ver si para pero desafortunadamente no corre, primero se detiene en un punto donde no pusimos ningún BP (en :004) y después queda colgado así:

```
Access violation when writing to [00000000] - use Shift+F7/F8/F9 to pass exception I   Running
```

Y no ocurre nada, somos cabezones y lo intentamos otra vez pero en esta ocasión mediante un "BP on Memory access" a ver qué pasa, apretamos F9 y tras pasar por el aluvión de excepciones salta un mensaje de error en Olly:



A ver si hay suerte, lo aceptamos e intentamos superar la excepción mediante Shift+F9 pero ni con esas, Olly no la puede manejar bien y desiste:

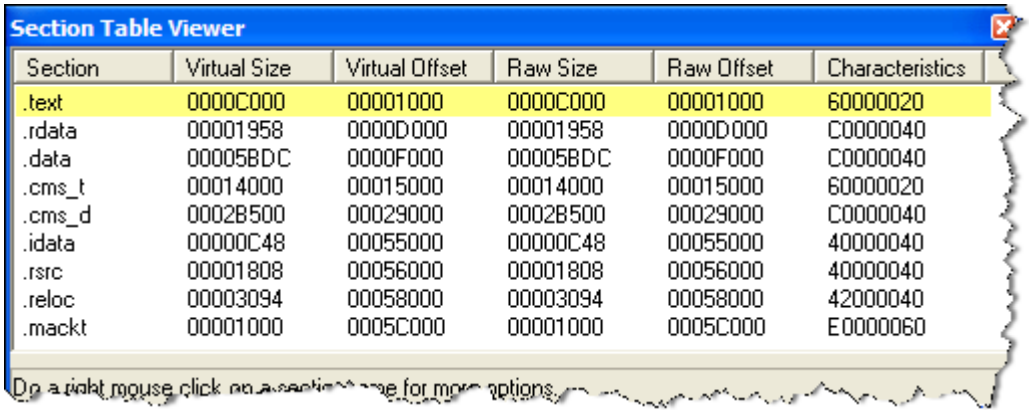
Si aprieto F9 una vez más el programa se cierra ☹

Ante esta situación más vale que nos vayamos despidiendo de los BPs comunes así como de los HBPs y los BPM on Memory, o sea que estamos mutilados, jeje… aquí se plantea el primer problema a superar, o bien localizamos y estudiamos el SEH que nos detecta o bien ingeniamos algo que nos detenga el pleno OEP, mi primer impulso fue el de intentar llegar al OEP con cualquier otro método, tampoco sirve el de las excepciones de Ricardo porque hace uso de los BPM on Memory que tampoco podemos utilizar.

Tirando de recursos :)

El siguiente objetivo está en llegar al OEP como sea!! No importa cómo la cuestión es llegar hasta él, no lo suelo hacer pero voy a echarle un vistazo con PEiD a ver qué me dice, después nosotros lo buscaremos igual pero bueno:

Tenemos una referencia, por la dirección que apunta es posible que sea el correcto pues cae dentro de la sección de código:



Section Table Viewer

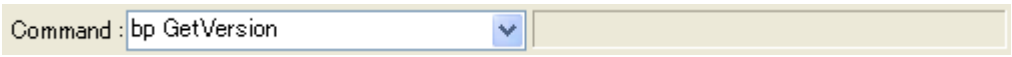
Section	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
.text	0000C000	00001000	0000C000	00001000	60000020
.rdata	00001958	0000D000	00001958	0000D000	C0000040
.data	00005BDC	0000F000	00005BDC	0000F000	C0000040
.cms_t	00014000	00015000	00014000	00015000	60000020
.cms_d	0002B500	00029000	0002B500	00029000	C0000040
.idata	00000C48	00055000	00000C48	00055000	40000040
.rsrc	00001808	00056000	00001808	00056000	40000040
.reloc	00003094	00058000	00003094	00058000	42000040
.mackt	00001000	0005C000	00001000	0005C000	E0000060

Do a right mouse click on a section name for more options...

Ya queda menos, jeje… lo siguiente que hice fue poner un BP en una api de las que sabemos que muchos compiladores hacen uso en sus primeras líneas justo después del OEP, varias son las candidatas, tenemos por ejemplo:

1.- GetVersion
2.- GetCommandLineA
3.- GetModuleHandleA
4.- etc...

Otra posible podría ser InitCommonControls pero en este caso atacaremos con GetVersion a ver si tenemos fortuna, colocamos el BP en el CommandBar:



Ojo con respetar mayúsculas y minúsculas, acto seguido apretamos F9 para que corra el programa y para en dicha api, miramos en la pila (stack) y vemos esto:

Address	Value	Comment
0012FF90	00420D29	RETURN to Game.00420D29 from kernel32.GetVersion
0012FF94	77D121AE	USER32.77D121AE
0012FF98	0012CD34	
0012FF9C	7FFDF000	
0012FFA0	F6D23C00	
0012FFA4	00000000	
0012FFA8	0012FF94	
0012FFAC	EA310045	
0012FFB0	0012FFE0	Pointer to next SEH record
0012FFB4	0042391C	SE handler
0012FFB8	0044ECF8	Game.0044ECF8
0012FFBC	FFFFFFFF	

Retornaremos a :00420d29 que cae dentro de la sección del packer así que no nos interesa, apretamos F9 de nuevo y queda un rato loopeando en la zona de las excepciones que ya conocemos de antes, esperamos que pase el vendaval y seguimos apretando F9 cada vez que pare si vemos en la pila una dirección que no caiga dentro de la sección de código, a la cuarta va la vencida:

Address	Value	Comment
0012F734	0040235C	RETURN to Game.0040235C from Game.00418AE0
0012F738	00403000	Game.00403000
0012F73C	00000009	
0012F740	7FFDF000	
0012F744	0012F77C	
0012F748	0012FFB0	
0012F74C	77F69005	ntdll.77F69005
0012F750	77F5D6B0	ntdll.77F5D6B0
0012F754	FFFFFFFF	
0012F758	77F4282E	RETURN to ntdll.77F4282E from ntdll.RtlLeaveCriticalSe
0012F75C	77FB3100	ntdll.77FB3100
0012F760	00000009	

Ahora si retorna a dicha sección, parece que vamos a tener suerte, quitamos el BP y pulsamos Ctrl+F9 para retornar, veamos donde caemos:

Address	Hex dump	Disassembly		Comment
00402330	55	push	ebp	Este es nuestro OEP ;)
00402331	8BEC	mov	ebp,esp	
00402333	6A FF	push	-1	
00402335	68 50D24000	push	0040D250	
0040233A	68 843D4000	push	00403D84	
0040233F	64:A1 00000000	mov	eax,fs:[0]	
00402345	50	push	eax	
00402346	64:8925 0000000	mov	fs:[0],esp	
0040234D	83EC 58	sub	esp,58	Se ha de reparar!
00402350	53	push	ebx	
00402351	56	push	esi	
00402352	57	push	edi	
00402353	8965 E8	mov	[ebp-18],esp	
00402356	FF15 00A34200	call	[42A300]	Game.00418AE0
0040235C	33D2	xor	edx,edx	
0040235E	8AD4	mov	dl,ah	
00402360	8915 F8344100	mov	[4134F8],edx	

Pues ese es nuestro OEP, como decía PEiD estaba en :00402330 podríamos dumpear desde aquí mismo pero yo soy partidario de hacerlo desde el mismo OEP, el problema es que las apis citadas entre las que se encuentra GetVersion nos dejan una vez pasado éste y no nos vale, debemos buscar otro método del que nos podamos valer para posicionarnos sobre el mismo OEP sin utilizar ningún tipo de BP, a priori puede parecer imposible pero creedme que no es así xDD

Buscando un método…

Bueno más que un método yo diría un modo, lo primero que se me ocurrió fue poner BPs en las apis como hicimos antes, pero ¿en cuál? Necesitamos una que sea invocada poco antes del punto de entrada, en un principio probé con GetProcAddress pero no me valía pues aparte de que se llamaba muchas veces me dejaba o mucho antes o ya después del OEP, lo mismo me ocurría con GetModuleHandleA y similares, aquí recordé que muchos packers ubicaban nuevas secciones de memoria para el desempakado mediante VirtualAlloc y al terminar el trabajo las liberaban con VirtualFree, así que pensé que este no sería una excepción, y así fue, pongamos un BP en dicha api:

```
Command : BP VirtualFree
```

Apretamos Enter para que el BP quede puesto y pulsamos F9 para que corra el programa, Olly se detiene, si miramos la pila veremos que retornaremos a una sección creada en tiempo de carga por el packer:

```
Address   Value     Comment
0012F624  2000219A  CALL to VirtualFree from 20002198
0012F628  014D0000  Address = 014D0000
0012F62C  00100000  Size = 100000 (1048576.)
0012F630  00004000  FreeType = MEM_DECOMMIT
0012F634  00000000
0012F638  00000000
0012F63C  0012F664
0012F640  20000000
0012F644  20001E96  RETURN to 20001E96 from 2000216A
0012F648  20001F11  RETURN to 20001F11 from 20001DE4
```

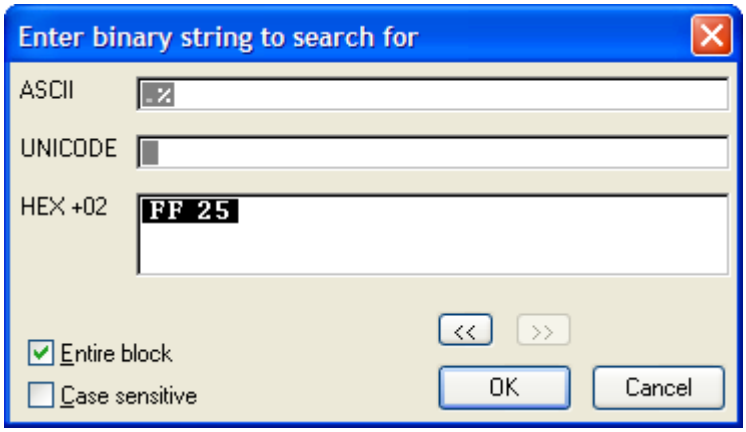
El retorno nos llevará a la sección :20000000 como digo creada en tiempo de carga pues en el Entry Point no existe, esto da igual si seguimos apretando Ctrl+F9 desde aquí llegará un momento en que llegaremos a zonas del packer:

```
Address   Hex dump     Disassembly                  Comment
004154E7  8B0D C41B4500  mov   ecx,[451BC4]
004154ED  51             push  ecx
004154EE  FF15 A0F64400  call  [44F6A0]             kernel32.CloseHandle
004154F4  833D D0F24400 0 cmp  dword ptr [44F2D0],0
004154FB  74 15          je    short 00415512
004154FD  6A 00          push  0
004154FF  FF15 2CFC4400  call  [44FC2C]
00415505  83C4 04        add   esp,4
00415508  C705 D0F24400 0 mov  dword ptr [44F2D0],0
00415512  833D D4F24400 0 cmp  dword ptr [44F2D4],0
00415519  74 20          je    short 0041553B
0041551B  833D D8F24400 0 cmp  dword ptr [44F2D8],0
00415522  74 0D          je    short 00415531
00415524  8B15 D8F24400  mov   edx,[44F2D8]
0041552A  52             push  edx
0041552B  FF15 A8F64400  call  [44F6A8]             kernel32.FreeLibrary
00415531  C705 D4F24400 0 mov  dword ptr [44F2D4],0
0041553B  837D 08 2C     cmp   dword ptr [ebp+8],2C
0041553F  74 50          je    short 00415591
00415541  837D 08 0C     cmp   dword ptr [ebp+8],0C
00415545  7F 4A          jg    short 00415591
00415547  C745 FC 7E0000 mov   dword ptr [ebp-4],7E
```

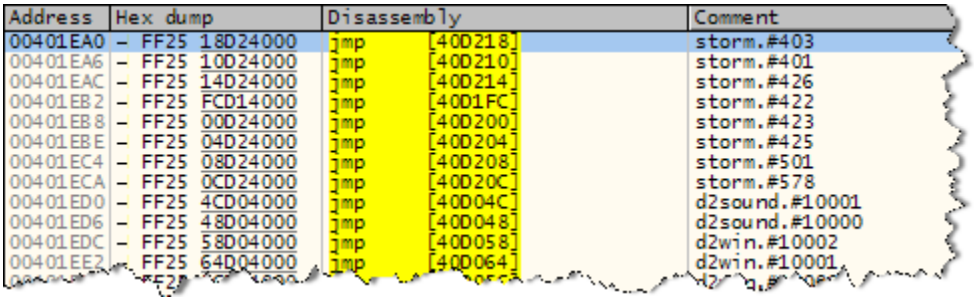
La llamada original era a FreeLibrary que es de donde venimos, si traceamos un poquitín más desde aquí llegaremos al salto al OEP ;)

```
Address   Hex dump     Disassembly                  Comment
00420005  90           nop
00420006  90           nop
00420007  50           push  eax
00420008  EB 03        jmp   short 0042000D
0042000A  58           pop   eax
0042000B  FFE0         jmp   eax                    Game.00402330
0042000D  A3 08174500  mov   [451708],eax
00420012  E8 6981FFFF  call  00418180
00420017  C785 BCFBFFFF 0 mov dword ptr [ebp-444],0
00420021  8B85 BCFBFFFF  mov   eax,[ebp-444]
00420027  6BC0 0A        imul  eax,eax,0A
```

Ese "jmp eax" nos lleva al mismo, es un primer paso este, una vez en él ya podemos dumpear, pero antes busquemos la tabla, una manera sencilla es pulsar Ctrl+B y en el cuadro que aparece buscar los FF15 ó FF25 que son los saltos y llamadas a la IAT:

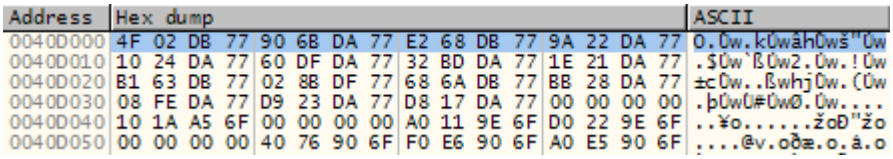


Aceptamos y caemos en la JMP_Table que es la tabla de saltos en cristiano:



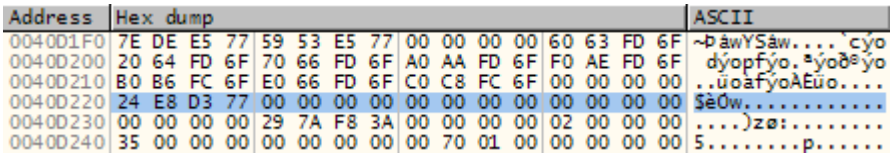
Address	Hex dump	Disassembly	Comment
00401EA0	FF25 18D24000	jmp [40D218]	storm.#403
00401EA6	FF25 10D24000	jmp [40D210]	storm.#401
00401EAC	FF25 14D24000	jmp [40D214]	storm.#426
00401EB2	FF25 FCD14000	jmp [40D1FC]	storm.#422
00401EB8	FF25 00D24000	jmp [40D200]	storm.#423
00401EBE	FF25 04D24000	jmp [40D204]	storm.#425
00401EC4	FF25 08D24000	jmp [40D208]	storm.#501
00401ECA	FF25 0CD24000	jmp [40D20C]	storm.#578
00401ED0	FF25 4CD04000	jmp [40D04C]	d2sound.#10001
00401ED6	FF25 48D04000	jmp [40D048]	d2sound.#10000
00401EDC	FF25 58D04000	jmp [40D058]	d2win.#10002
00401EE2	FF25 64D04000	jmp [40D064]	d2win.#10001

Seleccionamos uno con el botón derecho y damos a "Follow in dump->Memory address" para caer en plena tabla, desde aquí lo subimos un poco haciendo un scroll con el ratón o bien con los cursores para localizar el inicio de tabla:



Address	Hex dump	ASCII
0040D000	4F 02 DB 77 90 6B DA 77 E2 68 DB 77 9A 22 DA 77	O.Ûw.kÚwâhÛwš"Úw
0040D010	10 24 DA 77 60 DF DA 77 32 BD DA 77 1E 21 DA 77	.$Úw`ßÚw2.Úw.!Úw
0040D020	B1 63 DB 77 02 8B DF 77 68 6A DB 77 BB 28 DA 77	±cÛw..ßwhjÛw.(Úw
0040D030	08 FE DA 77 D9 23 DA 77 D8 17 DA 77 00 00 00 00	.þÚwÙ#ÚwØ.Úw....
0040D040	10 1A A5 6F 00 00 00 00 A0 11 9E 6F D0 22 9E 6F	..¥o.....žoÐ"žo
0040D050	00 00 00 00 40 76 90 6F F0 E6 90 6F A0 E5 90 6F	....@v.oðæ.o.å.o

En este caso es bien sencillo porque coincide con el inicio de la sección .rdata, hagamos lo mismo con el final de la misma:

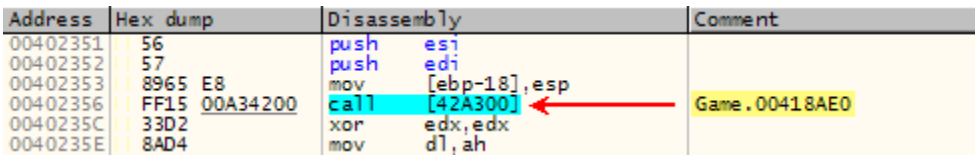


Address	Hex dump	ASCII
0040D1F0	7E DE E5 77 59 53 E5 77 00 00 00 00 60 63 FD 6F	~Þáw YSáw....`cýo
0040D200	20 64 FD 6F 70 66 FD 6F A0 AA FD 6F F0 AE FD 6F	dýopfýo.ªýoð®ýo
0040D210	B0 B6 FC 6F E0 66 FD 6F C0 C8 FC 6F 00 00 00 00	..üoàfýoÀÈüo....
0040D220	24 E8 D3 77 00 00 00 00 00 00 00 00 00 00 00 00	$èÓw.....
0040D230	00 00 00 00 29 7A F8 3A 00 00 00 00 02 00 00 00	....)zø:.....
0040D240	35 00 00 00 00 00 00 00 00 70 01 00 00 00 00 00	5.....p.....

Con estos dos valores obtenemos su longitud (final-inicio) quedando:

Largo: 40d220 - 40d000 == 220h

Pero antes de continuar veamos estas llamadas:



Address	Hex dump	Disassembly	Comment
00402351	56	push esi	
00402352	57	push edi	
00402353	8965 E8	mov [ebp-18],esp	
00402356	FF15 00A34200	call [42A300]	Game.00418AE0
0040235C	33D2	xor edx,edx	
0040235E	8AD4	mov dl,ah	

Puede parecer que son correctas pues apuntan a una rutina interna al ejecutable localizada en la sección .cms_t donde hay código del packer, pero en realidad se deben reparar porque a mitad de rutina buscan valores en secciones que no tiene el dumpeado:

```
Address   Hex dump    Disassembly          Comment
00418EA7  8B55 F4     mov   edx,[ebp-C]
00418EAA  3311        xor   edx,[ecx]
00418EAC  8955 F4     mov   [ebp-C],edx
00418EAF  E9 ED000000 jmp   00418FA1
00418EB4  A1 90F74400 mov   eax,[44F790]
```

En esa posición intenta leer una sección inexistente:

```
ds:[013B0970]=???
edx=510D8D6B (DSOUND.510D8D6B)
```

Si traceamos la rutina desde el ejecutable original empakado veremos el valor que lee:

```
ds:[013B0970]=514D384C
edx=510D8D6B (DSOUND.510D8D6B)
```

Esto es un problema, además no es única pues hay otras direcciones en las que se accede a posiciones de dicha sección :013b0000 que tiene un tamaño de 1000h bytes, si tuviéramos la suerte de pasar todas llegaríamos a la verdadera api:

```
Address   Hex dump   Disassembly          Comment
0041926E  8B06      mov   eax,[esi]
00419270  5F        pop   edi
00419271  5E        pop   esi
00419272  5B        pop   ebx
00419273  8BE5      mov   esp,ebp
00419275  5D        pop   ebp
00419276  FFE0      jmp   eax            kernel32.GetVersion
00419278  5F        pop   edi
00419279  5E        pop   esi
0041927A  5B        pop   ebx
0041927B  8BE5      mov   esp,ebp
0041927D  5D        pop   ebp
0041927E  C3        ret
```

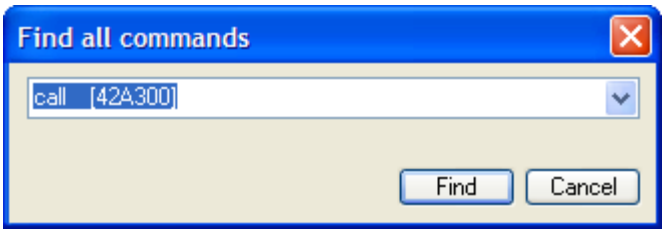
Al llegar al RET volverá al ejecutable, por lo menos ya sabemos cómo obtener la api asociada a cada call de estos, cosa necesaria para reconstruir el .exe antes de dumpear, tenemos dos opciones como mínimo, bueno tres, una dumpear como está y volcar también el bloque de memoria :013b0000 para luego cargarlo mediante VirtualAlloc en el dumpeado para que corra (poco elegante), otra sería currarnos un injerto que nos recupere los calls como originalmente eran antes de pasarle el packer (muy elegante) y una tercera programarnos un script que haga este mismo trabajo (igual de elegante), optamos por esta tercera opción, manos a la obra xDD

Programando un script!

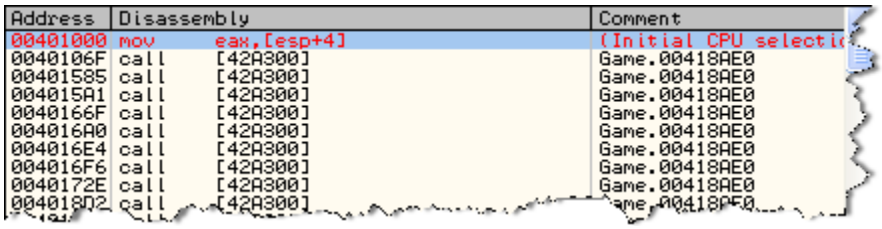
Pues así es, en estos casos es una buena solución y programarlo es una tarea muy sencilla si se sabe claramente lo que debe hacer, en este caso los pasos serán estos:

1.- Irá recorriendo una tabla donde estarán las direcciones de todos los calls a reparar, luego veremos cómo obtener dicha tabla
2.- Ejecutará el call apuntado y obtendrá la api correcta
3.- realizará una búsqueda en la iat de la api encontrada
4.- reconstruirá el call de modo que apunte a la iat y quede reparado

Esa es la secuencia de pasos que realizará dicho script, la tabla la obtenemos de la siguiente forma: nos vamos al inicio de la sección de código que está en :401000 y lanzamos una búsqueda con "Search for->All commands" del call a reparar:

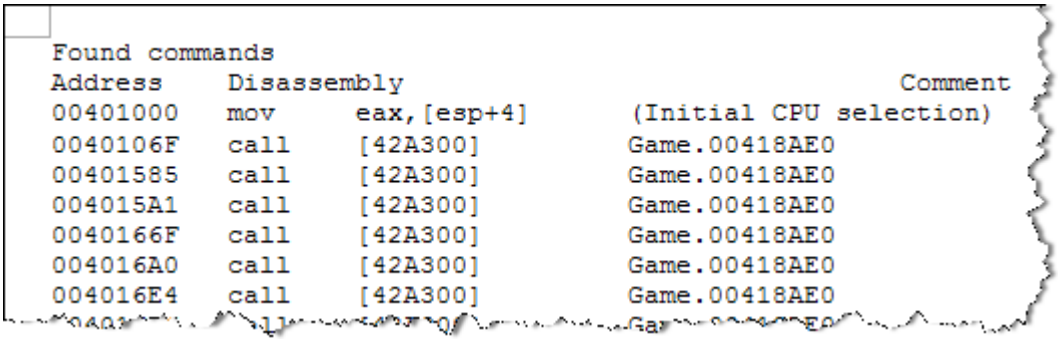


Pulsamos "Find" y nos hará un listado con todos ellos en la ventana de referencias:



Address	Disassembly		Comment
00401000	mov	eax,[esp+4]	(Initial CPU selecti
0040106F	call	[42A300]	Game.00418AE0
00401585	call	[42A300]	Game.00418AE0
004015A1	call	[42A300]	Game.00418AE0
0040166F	call	[42A300]	Game.00418AE0
004016A0	call	[42A300]	Game.00418AE0
004016E4	call	[42A300]	Game.00418AE0
004016F6	call	[42A300]	Game.00418AE0
0040172E	call	[42A300]	Game.00418AE0
004018D2	call	[42A300]	Game.00418AE0

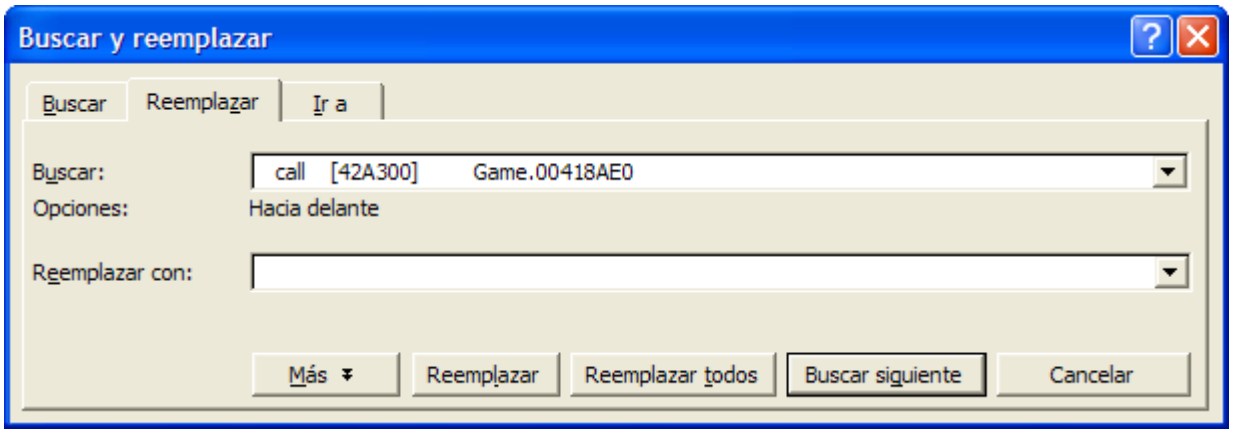
Salvamos esta tabla con "Copy to clipboard->Whole table" abrimos el Word y creamos un nuevo documento que renombraremos a calls.doc y pegamos en él la tabla completa de los calls a reparar, quedará algo así:



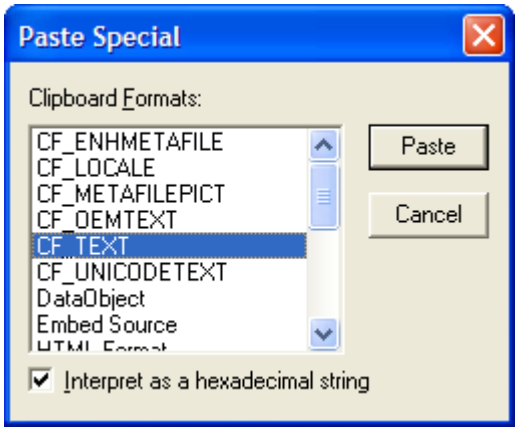
Found commands

Address	Disassembly		Comment
00401000	mov	eax,[esp+4]	(Initial CPU selection)
0040106F	call	[42A300]	Game.00418AE0
00401585	call	[42A300]	Game.00418AE0
004015A1	call	[42A300]	Game.00418AE0
0040166F	call	[42A300]	Game.00418AE0
004016A0	call	[42A300]	Game.00418AE0
004016E4	call	[42A300]	Game.00418AE0

Este formato no es correcto, sólo nos interesa la dirección de los calls, así que cambiaremos la cadena "call [42A300] Game.00418AE0 " por espacios blancos:



En total hay 193 calls que hay que reparar, los intentaré reparar como hicimos con el ASProtect 2.0x, hacemos un binary copy con Ctrl+C en toda la tabla de direcciones que han quedado en el documento calls.doc (después de eliminar toda la basura), abrimos el HexWorkShop v4.23 aunque otra versión también vale, creamos un nuevo documento Ctrl+N y vamos al menú "Edit->Paste Special…" y lo dejamos así:



Aceptamos y quedará el documento casi terminado, sólo queda ir al menú "Tools" y elegir la opción "Operations->Byte Flip…" y marcamos "32 Bit Unsigned Long" en el campo "Thread Data As" en "Byte Ordering" ponemos "Little Endien" si nos da la opción, si no la da pues nada, aceptamos y ya tenemos la tabla que necesitaremos para el script más tarde, la salvamos como calls_ok.doc o en un .txt cuestión de gustos.

Una vez tenemos la tabla buscamos un lugar seguro para pegarla dentro del ejecutable que tenemos parado en el OEP, encontramos uno muy bueno en :0040C600

Ahora que tenemos esta parte terminada nos podemos preocupar del script, primero os muestro como quedó y después lo explico, aunque ya está bien detallado en los comentarios que agrego al lado de cada instrucción:
El script es éste y está hecho con ConTEXT v0.98.3 también podríamos haber utilizado el editor de Arapumk que va también fenomenal esto es lo de menos, la cuestión es que nos quede algo así:

```
var posIAT
var api
var tabla
var puntero
var direc

mov tabla, 40c600        // nos situamos en la tabla de calls_rotos

inicio:
mov ecx, [tabla]         // cogemos la dirección del call_roto a reparar
cmp ecx, 0               // miramos si terminó el trabajo
je termina               // si es así termina el script
mov direc, ecx           // salvamos la dirección del call a reparar
log direc
mov eip, ecx             // sino situamos allí el EIP
eob vamos                // si salta un BP o una excepción salta a vamos"
run                      // dejamos correr el programa

vamos:
cmp eip, 00419276        // mira si tenemos ya la api en EAX
jne termina              // si no es así termina el trabajo del script
mov api, eax             // salvamos la api en nuestra variable
log api                  // logueamos este valor
jmp localizar            // y vamos a localizar la api en la tabla

localizar:
mov posIAT, 40d000       // situamos un puntero al inicio de la IAT
```

```
sigue:
mov edx, [posIAT]        // cogemos el contenido de la entrada
cmp edx, api             // es la misma api que la nuestra?
je tengo                 // si la tenemos vamos a "tengo"
add posIAT, 4            // apuntamos al siguiente DWORD
cmp 40d224, posIAT       // mira si hemos llegado al final de tabla
jb problemas             // si es así hay problemas con la api
jmp sigue                // sino seguimos loopeando...

tengo:
mov edx, direc           // recuperamos la dirección del call
add edx, 2               // apuntamos a los bytes a modificar
mov [edx], posIAT        // los cambiamos por los correctos
mov [tabla], 0           // borramos la dirección anterior
add tabla, 4             // apuntamos a la siguiente entrada
jmp inicio               // y vamos a por el siguiente

problemas:
MSG "Hay problemas con una api :("
ret

termina:
MSG "Enhorabuena ya tenemos iat¡!!!!"
ret
```

Es muy sencillo pero igual explicaré lo más importante, antes de nada decir que para funcionar necesita que haya un HBP on Execution en la dirección del "JMP EAX" cada uno que ponga el suyo, recordemos cual es en mi máquina:

```
00419273    8BE5        mov     esp,ebp
00419275    5D          pop     ebp
00419276    FFE0        jmp     eax
```

Está en :00419276 y allí irá el HBP como se puede ver en el script, entonces su funcionamiento es el siguiente, os explico:

```
var posIAT
var api
var tabla
var puntero
var direc

mov tabla, 40c600        // nos situamos en la tabla de calls_rotos
```

Aquí únicamente se declaran las variables que vamos a utilizar, en este caso son 5, después se inicializa la variable "tabla" para que apunte al inicio de la tabla de calls a reparar (o calls_rotos como decía ZettK en su tute de ASProtect 2.0x)

```
inicio:
mov ecx, [tabla]         // cogemos la dirección del call_roto a reparar
cmp ecx, 0               // miramos si terminó el trabajo
je termina               // si es así termina el script
mov direc, ecx           // salvamos la dirección del call a reparar
log direc
mov eip, ecx             // sino situamos allí el EIP
eob vamos                // si salta un BP o una excepción salta a vamos"
run                      // dejamos correr el programa
```

Aquí comienza realmente el script, esto es un loop donde se recorre la tabla de calls a reparar situando allí el EIP en cada pasada para ejecutar dicho call, la condición de parada es cuando el valor de la tabla sea igual a 0.

```
vamos:
cmp eip, 00419276        // mira si tenemos ya la api en EAX
jne termina              // si no es así termina el trabajo del script
mov api, eax             // salvamos la api en nuestra variable
log api                  // logueamos este valor
jmp localizar            // y vamos a localizar la api en la tabla
```

Etiqueta "vamos", aquí se mira si la parada fue a causa del HBP que tenemos puesto en el "JMP EAX" si es el caso salvamos el valor de EAX (que contiene la api correcta) en la variable "api" para uso posterior y además la loguea, la función del Log es por informar de lo que va haciendo el script y ver qué apis va tomando, así si hay algún error sabremos exactamente en qué entrada ocurrió con sólo mirar el Log.

```
localizar:
mov posIAT, 40d000       // situamos un puntero al inicio de la IAT

sigue:
mov edx, [posIAT]        // cogemos el contenido de la entrada
cmp edx, api             // es la misma api que la nuestra?
je tengo                 // si la tenemos vamos a "tengo"
add posIAT, 4            // apuntamos al siguiente DWORD
cmp 40d224, posIAT       // mira si hemos llegado al final de tabla
jb problemas             // si es así hay problemas con la api
jmp sigue                // sino seguimos loopeando...
```

Este procedimiento es el encargado de localizar la api encontrada en la iat y salvar su posición para su posterior parcheo, necesitamos la dirección de inicio y final de tabla:

```
tengo:
mov edx, direc           // recuperamos la dirección del call
add edx, 2               // apuntamos a los bytes a modificar
mov [edx], posIAT        // los cambiamos por los correctos
mov [tabla], 0           // borramos la dirección anterior
add tabla, 4             // apuntamos a la siguiente entrada
jmp inicio               // y vamos a por el siguiente
```

El procedimiento "tengo" se encarga de reconstruir los calls redirigiéndolos hacia la iat, de manera que apunten a la api en lugar de a la rutina de SecuROM.

```
problemas:
MSG "Hay problemas con una api :("
ret

termina:
MSG "Enhorabuena ya tenemos iat¡!!!!"
ret
```

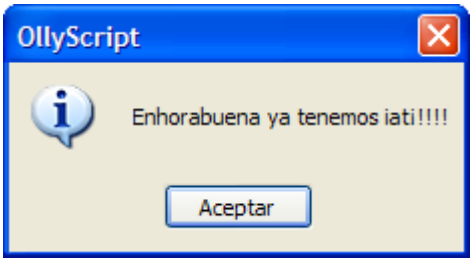
Esto último son los mensajes preparados, el primero "problemas" será mostrado en caso que la iat no exista en la tabla y el segundo cuando termine el script.

Dicho esto sólo nos queda ejecutarlo, tenemos el programa detenido en el OEP, vamos a :0040C600 y pegamos la tabla que tenemos preparada:

Address	Hex dump	ASCII
0040C600	6F 10 40 00 85 15 40 00 A1 15 40 00 6F 16 40 00	o.@...@...@.o.@
0040C610	A0 16 40 00 E4 16 40 00 F6 16 40 00 2E 17 40 00	..@.ä.@.ö.@...@
0040C620	D2 18 40 00 DD 18 40 00 FF 18 40 00 1C 19 40 00	Ò.@.Ý.@...@...@
0040C630	B4 19 40 00 EF 19 40 00 84 1B 40 00 CC 1B 40 00	..@.ï.@...@.Ì.@
0040C640	3C 1C 40 00 58 1C 40 00 78 1C 40 00 7F 1C 40 00	<.@.X.@.x.@...@
0040C650	AE 1C 40 00 E5 1C 40 00 FF 1C 40 00 47 1D 40 00	®.@.å.@...@.G.@
0040C660	4E 1D 40 00 7C 1D 40 00 87 1D 40 00 8E 1D 40 00	N.@.|.@...@.Ž.@
0040C670	B3 1D 40 00 05 1E 40 00 6D 1E 40 00 8C 1E 40 00	³.@...@.m.@.Œ.@
0040C680	56 23 40 00 B6 23 40 00 E1 23 40 00 04 24 40 00	V#@..¶#@.á#@..$@
0040C690	7A 24 40 00 5C 2E 40 00 84 2E 40 00 95 2E 40 00	z$@.\.@...@...@
0040C6A0	BA 2E 40 00 DD 2E 40 00 EB 2E 40 00 0F 2F 40 00	º.@.Ý.@.ë.@../@
0040C6B0	20 2F 40 00 37 2F 40 00 5A 2F 40 00 DA 2F 40 00	/@.7/@.Z/@.Ú/@
0040C6C0	3E 31 40 00 45 31 40 00 C6 31 40 00 2E 33 40 00	>1@.E1@.Æ1@..3@
0040C6D0	A6 34 40 00 FF 36 40 00 9E 37 40 00 B1 37 40 00	¦4@..6@.ž7@.±7@
0040C6E0	EF 37 40 00 60 38 40 00 0B 39 40 00 6E 39 40 00	ï7@.`8@..9@.n9@
0040C6F0	7C 39 40 00 B3 39 40 00 E1 39 40 00 1C 3A 40 00	|9@.³9@.á9@..:@
0040C700	5E 3A 40 00 93 3A 40 00 F3 3A 40 00 98 3B 40 00	^:@..:@.ó:@..;@
0040C710	D7 3B 40 00 71 3C 40 00 83 3C 40 00 02 3F 40 00	×;@.q<@..<@..?@
0040C720	D8 3F 40 00 DF 3F 40 00 58 40 40 00 72 40 40 00	Ø?@.ß?@.X@@.r@@
0040C730	A6 40 40 00 DE 40 40 00 34 41 40 00 46 41 40 00	¦@@.Þ@@.4A@.FA@
0040C740	F1 41 40 00 FE 41 40 00 66 43 40 00 B3 43 40 00	ñA@.þA@.fC@.³C@
0040C750	CB 43 40 00 E0 43 40 00 7B 45 40 00 9E 45 40 00	ËC@.àC@.{E@.žE@
0040C760	CD 45 40 00 F0 45 40 00 87 46 40 00 1D 47 40 00	ÍE@.ðE@..F@..G@
0040C770	1B 49 40 00 34 4A 40 00 4F 4A 40 00 65 [illegible]	.I@.4J@.OJ@.e[illegible]

Llega más para abajo, justo hasta :0040C8FF.

Ahora lanzamos el script sin miedo desde "Plugins->OllyScript->Run script…" unos minutos después aparece este mensaje:



Loa aceptamos y sale el otro que cerramos igual, ya tenemos el trabajo terminado, si echamos un vistazo a las llamadas veremos que están ya reparadas:

Address	Hex dump	Disassembly		Comment
00402330	55	push	ebp	
00402331	8BEC	mov	ebp,esp	
00402333	6A FF	push	-1	
00402335	68 50D24000	push	0040D250	
0040233A	68 843D4000	push	00403D84	
0040233F	64:A1 00000000	mov	eax,fs:[0]	
00402345	50	push	eax	
00402346	64:8925 000000(	mov	fs:[0],esp	Importación reparada :)
0040234D	83EC 58	sub	esp,58	
00402350	53	push	ebx	
00402351	56	push	esi	
00402352	57	push	edi	
00402353	8965 E8	mov	[ebp-18],esp	
00402356	FF15 D4D04000	call	[40D0D4]	kernel32.GetVersion
0040235C	33D2	xor	edx,edx	
0040235E	8AD4	mov	dl,ah	
00402360	8915 F8344100	mov	[4134F8],edx	

Ese call antes apuntaba a la rutina del packer y ahora queda reparado apuntando directamente a GetVersion que es el lugar adonde apuntaba antes de ser empakado xDD

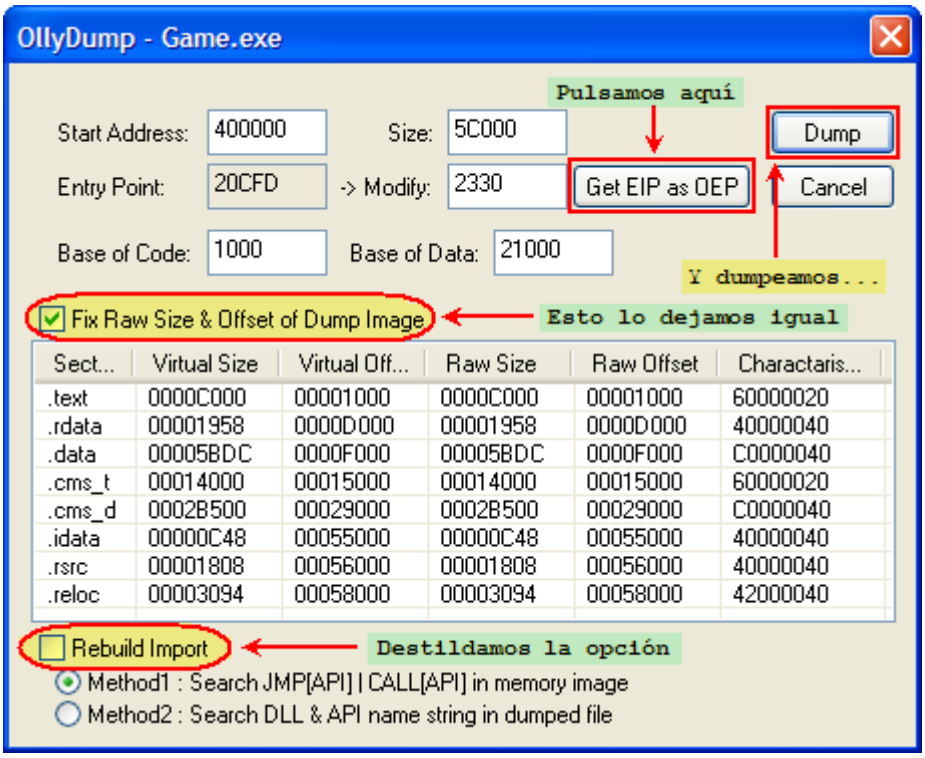
Podemos ir mirando por el código para darnos cuenta que están todos arreglados:

Address	Hex dump	Disassembly		Comment
004023AE	8975 FC	mov	[ebp-4],esi	
004023B1	E8 4C140000	call	00403802	
004023B6	FF15 2CD14000	call	[40D12C]	kernel32.GetCommandLineA
004023BC	A3 D84B4100	mov	[414BD8],eax	
004023C1	E8 0A130000	call	004036D0	
004023C6	A3 DC344100	mov	[4134DC],eax	
004023CB	E8 B3100000	call	00403483	
004023D0	E8 F50F0000	call	004033CA	
004023D5	E8 E20C0000	call	004030BC	
004023DA	8975 D0	mov	[ebp-30],esi	
004023DD	8D45 A4	lea	eax,[ebp-5C]	
004023E0	50	push	eax	
004023E1	FF15 28D14000	call	[40D128]	kernel32.GetStartupInfoA
004023E7	E8 860F0000	call	00403372	
004023EC	8945 9C	mov	[ebp-64],eax	
004023EF	F645 D0 01	test	byte ptr [ebp-30],1	
004023F3	74 06	je	short 004023FB	
004023F5	0FB745 D4	movzx	eax,word ptr [ebp-2C	
004023F9	EB 03	jmp	short 004023FE	
004023FB	6A 0A	push	0A	
004023FD	58	pop	eax	
004023FE	50	push	eax	
004023FF	FF75 9C	push	dword ptr [ebp-64]	
00402402	56	push	esi	
00402403	56	push	esi	
00402404	FF15 D0D04000	call	[40D0D0]	kernel32.GetModuleHandleA
0040240A	50	push	eax	

Antes de dumpear situamos el EIP en su lugar pues ahora está aquí:

Address	Hex dump	Disassembly		Comment
00419276	FFE0	jmp	eax	ntdll.RtlReAllocateHeap
00419278	5F	pop	edi	
00419279	5E	pop	esi	
0041927A	5B	pop	ebx	
0041927B	8BE5	mov	esp,ebp	
0041927D	5D	pop	ebp	
0041927E	C3	ret		

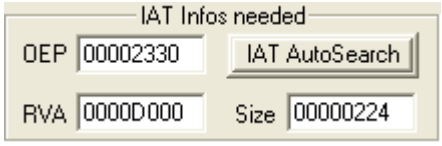
Vamos al OEP==00402330 y hacemos un "New origin here" después dumpeamos:



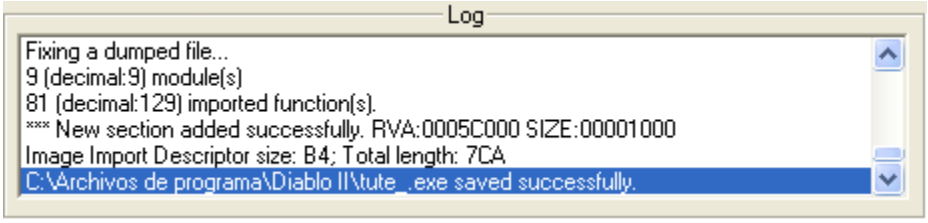
Las opciones son las mismas que utilizamos en un Armadillo o ASProtect, etc., nos pedirá un nombre, lo podemos llamar tute.exe por ejemplo.

Cuando tengamos listo el dumpeado abrimos el Import Reconstructor v1.6 de MackT/uCF, magnífica herramienta por cierto, seleccionamos el programa de

entre los procesos en ejecución desde la lista "Attach to an Active Process" y le damos los datos:



Supongo que no hará falta comentar de donde han salido, doy por hecho que se tienen unos mínimos conocimientos y además está muy bien documentado en otros tutes de los compañeros de la lista, pulsamos en "Get Imports" y nos salen 129 funciones importadas de las que no hay ni una sola incorrecta, sólo nos queda pulsar "Fix Dump" y darle el dumpeado tute.exe, al poco nos informa que el .exe fue reconstruido y ahora se llama tute_.exe que es el que tiene la iat reparada:

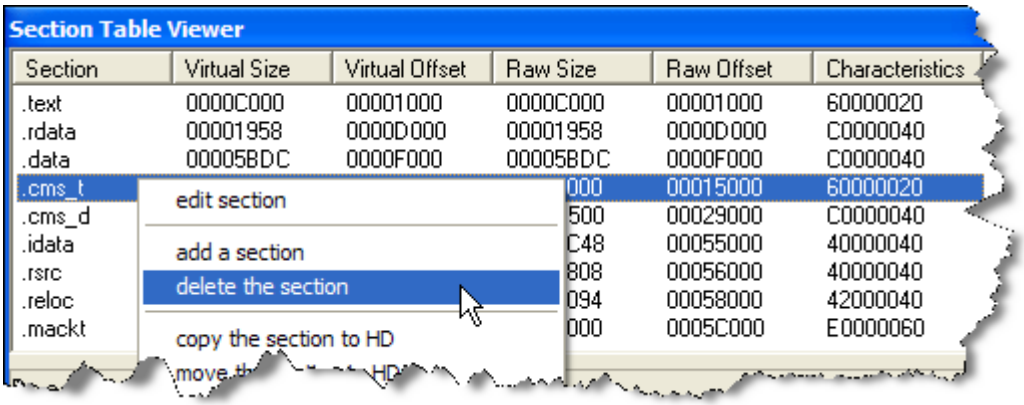


Vamos a la carpeta donde tenemos instalado el juego y lo ejecutamos…

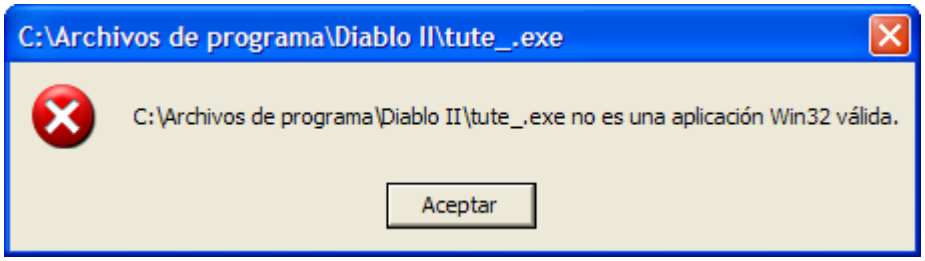
Allí está corriendo, si cerramos las DAEMON Tools corre igual, ya no depende de SecuROM, jeje… lo tenemos totalmente desempakado.

Además si miramos el ejecutable desprotegido veremos que ocupa 372Kb contra los 415Kb del original empakado, además algunas secciones del ejecutable quedan inutilizadas pudiéndose eliminar del mismo y reconstruir el .exe esto pasa con al menos tres de ellas como .cms_s, cms_d y .idata, para terminar os mostraré cómo disminuir el tamaño del ejecutable de los 372Kb actuales a tan sólo 78Kb.

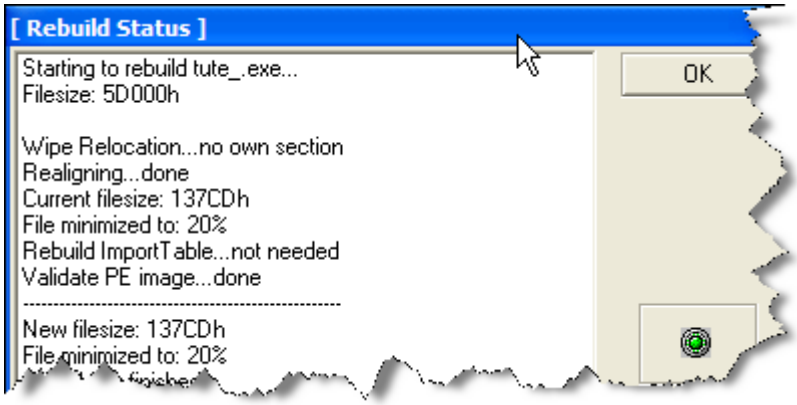
Abrimos PEditor v1.7 y vargamos tute_.exe vamos a sections y allí eliminamos las tres secciones que cité antes de esta manera:



Esto lo hacemos con cms_t, cms_d y con .idata, estas tres las eliminamos, una vez hecho si probamos a correr el ejecutable nos encontraremos esto:



Pero no nos asustemos, corremos LordPE y arrastramos sobre él, nos hará un Rebuild (reconstrucción) reduciendo el tamaño final del ejecutable, esto lo hacemos entre otras cosas porque hemos eliminado las secciones no útiles donde trabajaba el packer y que no nos harán falta de aquí en adelante, una vez terminado mostrará el Log tal que así:



Ya ha terminado, podemos ver que se ha reducido en un 20%, no es demasiado pero no está mal aunque en otras ocasiones podemos obtener un .exe reducido en un mayor porcentaje pero bueno tampoco está mal, ahora veamos cómo quedó:

[Section Table]

Name	VOffset	VSize	ROffset	RSize	Flags
.text	00001000	0000C000	00000200	0000B5A3	60000020
.rdata	0000D000	00002000	0000B800	00001958	C0000040
.data	0000F000	00047000	0000D200	000043AE	C0000040
.rsrc	00056000	00006000	00011600	00001805	40000040
.mackt	0005C000	00001000	00013000	000007CD	42000040

Estas son las secciones resultantes que se salvaron de la quema, si miramos el tamaño veremos que ocupa tan solo 78Kb en contra de los 414kB que originalmente tenía, lo que nos da un ratio de 81,15% de reducción, francamente perfecto, imaginemos que el ejecutable ocupe 12Mb, según estos cálculos obtendríamos uno de tan solo 2,262Mb, sin comentarios xDDD

Con esto terminamos el tute de hoy, espero que os haya gustado y que sirva de ayuda a alguien con el tema de desempakado, con eso me conformaría, un abrazo a todo el grupo y decir que mi vuelta es ya un hecho, espero escribir muchos tutes más. Un abrazo a TODOS¡!!!!

Un saludo también para "Gabriel" él ya sabe quien es ;)

Para cualquier cosa ya sabéis donde contactarme: syxe05@gmail.com XD